

MAST90106 Capstone: Data Format and Validation Registry

Group 19: Lei Cao, Xiaohan Deng, Xuanyi Zhang, Yaojin Wu, Zhizhou Zhu

May 2025

1 Introduction

Data format and validation registry is an industry project supported by AURIN. We aim to build an extensible validation system for its diversified data sets.

Urban data infrastructure covers a wide range of complex data sets. However, data formats from different sources may be heterogeneous and lack of unified standards, so it is difficult to find and correct data format and quality problems in time. The lack of a standardized verification framework is also detrimental to the integrity and interoperability of the data, making the data unable to fully meet the requirements of subsequent analysis and application.

Reliable spatial data is the basis of scientific decision-making and effective planning. Therefore, ensuring the consistency of data format and the accuracy of content is crucial for data analysis and scientific decision-making. Improving data quality through automated validation processes will significantly promote the improvement of urban infrastructure decision-making, optimization and robustness of resource allocation.

From the perspective of data science, there are many challenges to overcome, including:

- **Large amount of data and various formats:** The spatial data of different platforms may be large in scale, covering a variety of formats, and the structure and requirements of each format are different.
- **Complex spatial structure:** Many data contain complex geometric structures, such as 2D or 3D polygons, and the correctness of their geometric topological relationships needs to be ensured.
- **Semantic diversity:** The attribute fields of different data sets and their semantics differ significantly, and their context meanings need to be understood during validation.
- **Complicated verification rules:** Spatial data verification involves rules at multiple levels, such as geometry, topology and semantics, and needs comprehensive inspection.
- **Problem location and visualization:** It needs to accurately locate the source of the problem and visualize it for rapid diagnosis and repair.
- **Data repair and storage:** Convenient tools and interfaces shall be provided to facilitate and quickly mark and repair problems and ensure stable data storage, easy backtracking and subsequent calls.

To solve the above challenges, this project will design and implement an extensible automated data format and verification registration system. In this way, the format and quality verification of multi-source heterogeneous spatial data can be realized, and the overall quality and credibility of data can be improved finally.

2 Related Works

Recent development in geospatial data modelling has led to more research aimed at ensuring the consistency, accuracy, and reliability of spatial datasets. With the increase in data complexity and volume, the need for data validation methods has become crucial. This section highlights the key findings from the literature review, focusing on data format and registry validation, and pointing out gaps and opportunities in current approaches to spatial data quality assurance.

From the data validation perspective, we reviewed *Validating 3D Land Parcels in Victoria* (Shojaei et al., 2017), which proposed validation rules for data integrity, including volume consistency, no overlaps, flat surfaces, and closed shapes. All these rules utilized geometric calculations to improve the integrity and reliability of data. We also reviewed *Validating Massive Spatial Data with SpatialHadoop* (Migliorini, 2016), which addressed the challenge of validating massive spatial datasets using SpatialHadoop and proposed rules to ensure spatial consistency across geographic units of different scales. These include the ExistIn rule, DjSiblings Rule, and ComposedOf Rule. Another study, *A Structured Framework for 3D Cadastral Data Validation* (Asghari et al., 2020), developed a framework to improve the quality and reliability of land administration systems using rules for geometric validity, topological consistency, and semantic accuracy. *Keeping Shape Data Clean During Edits* (Bellet et al., 2011) introduced a graph-based model utilizing nodes and edges, with topological and attribute rules to manage shape data integrity.

From the data format point of view, *Semantic-Driven Parametric 3D Geographic Scene Modelling* (Dang et al., 2025) introduces a framework integrating Large Language Models with knowledge graphs to generate large 3D geographic scenes. It includes RDF knowledge graphs, LLMs with chain-of-thought reasoning, and a parametric modelling engine. The study *Application of Large Language Models in Geographic Map Analysis and Visualization* (Nguyen et al., 2025) presents a GIS system utilizing GPT-4o with a pipeline including query-structure generation, solution-graph modelling, automatic execution, and empirical validation. *Formalizing GeoJSON for Web-Based Spatial Data* (Butler et al., 2016) proposed using GeoJSON as a standard for web geospatial applications, examining its design, interoperability, and performance.

From the data registry perspective, *Validation Registries in SDIs* (Rajabifard et al., 2003) highlights the importance of validation registries in Spatial Data Infrastructures (SDIs), using an Australian case to demonstrate how registry architecture supports data quality. *A Review on Accuracy and Uncertainty of Spatial Data* (Devendran and Lakshmanan, 2014) outlines five key quality dimensions: data lineage, positional accuracy, attribute accuracy, logical consistency, and completeness. Finally, *Geospatial Data Quality (ISO 19157-1): Evolve or Perish* (Ariza López, 2020) identifies five improvement areas: extending quality elements, enhancing interoperability, improving quality measures, supporting streaming data, and enabling machine-readable metadata.

3 Data analysis

From last section, we learned that there's no one-size-fits-all method for validating geospatial data. Different types of data need different rules. For instance, geospatial data can differ in dimensionality, i.e., some dataset are two-dimensional while others are three-dimensional. Each format presents unique challenges and requires customized validation rules. As this was our first time working for geospatial data, we choose to begin with the simpler 2D datasets to familiarize ourselves with the processing workflow. Once we have more experience, we intend to extend our approach to include 3D datasets as well.

While the key is always to think logically, e.g., does this surface, soil, or property make sense given its context? For illustration, we conducted exploratory data analysis on multiple datasets with various natures and here we demonstrate a general pipeline for data validation through a specific case. The dataset shows the official suburb boundaries in South Australia as of August 2018 ([Government of South Australia and Infrastructure, 2018](#)). These boundaries defined under the Geographical Names Act 1991 for addressing purposes are based on cadastral, topographic, and road data. The data are stored in a GeoJSON file, where a sample entry is like:

```
{
  "type": "Feature",
  "id": "sa_suburb_boundaries_aug_2018.1",
  "geometry": {
    "type": "MultiPolygon",
    "coordinates": [[[
      [138.53137231, -34.83256411],
      ...,
      [138.53137231, -34.83256411]
    ]]]
  },
  "geometry_name": "wkb_geometry",
  "properties": {
    "ogc_fid": 1,
    "postcode": "5013",
    "suburb": "GILLMAN",
    "suburb_number": 501301,
    "legalstartdate": "2009-10-21T13:00:00Z",
    "shape_length": 0.125751231186375,
    "shape_area": 3.83556221183474E-4
  }
},
```

which contains a MultiPolygon object with coordinates in latitude and longitude as well as the semantic features corresponding to this object. We followed a generalised procedure, which contains three key aspects of validation rules-geometric validity, topological consistency, and semantic accuracy as mentioned in literature to have insights on what to check for geospatial data.

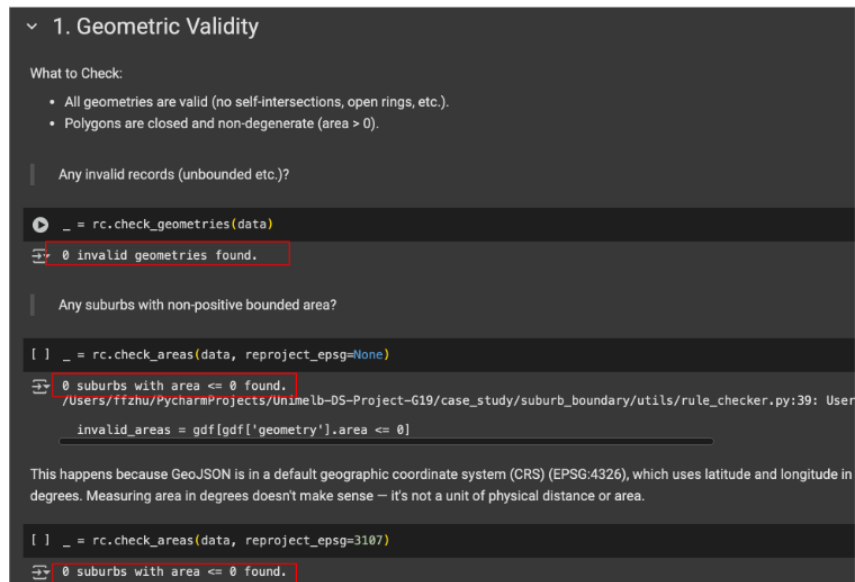
3.1 Geometric Validity

Geometric validation ensures that the dataset contains structurally correct and meaningful geometries. Two key checks were applied:

- **Geometry validity:** Verified that there are no self-intersections or unclosed rings.
- **Non-degenerate polygons:** Ensured all polygons have a positive area after reprojection.

It is noticeable that the original data were built under the default CRS (EPSG:4326) using global latitude-longitude system. Directly proceeding subsequent calculations under this schema will result in weird small values, and hence we should reproject the coordinates to a correct scale preferably in meters. Considering the nature of the dataset was about South Australian suburb boundaries, we reprojected them to EPSG:3107, which suited for South Australia to maintains accuracy in local spatial analysis.

The validation results shown in Figure 1 highlight that 0 invalid geometries and 0 suburbs with zero or negative area were found (warnings under default CRS and no warnings after reprojection to EPSG:3107).



```
1. Geometric Validity

What to Check:
• All geometries are valid (no self-intersections, open rings, etc.).
• Polygons are closed and non-degenerate (area > 0).

Any invalid records (unbounded etc.)?

_ = rc.check_geometries(data)
0 invalid geometries found.

Any suburbs with non-positive bounded area?

[ ] _ = rc.check_areas(data, reproject_epsg=None)
0 suburbs with area <= 0 found.
/Users/ffzhu/PycharmProjects/Unimelb-05-Project-619/case_study/suburb_boundary/utlis/rule_checker.py:39: UserWarning: Invalid area found.
  invalid_areas = gdf[gdf['geometry'].area <= 0]

This happens because GeoJSON is in a default geographic coordinate system (CRS) (EPSG:4326), which uses latitude and longitude in degrees. Measuring area in degrees doesn't make sense — it's not a unit of physical distance or area.

[ ] _ = rc.check_areas(data, reproject_epsg=3107)
0 suburbs with area <= 0 found.
```

Figure 1: Geometric Validity Checks

3.2 Topological Consistency

Topological validation checks how well the geometries fit together spatially, including ensuring no overlaps, no gaps, and accurate shared boundaries. Three rules were evaluated:

- No overlapping suburbs
- No topological gaps
- Consistent shared boundaries (no slivers or gaps)

Results shown in Figure 2 reveal 14 overlapping suburb pairs but no topological gaps or inconsistent shared boundaries were detected

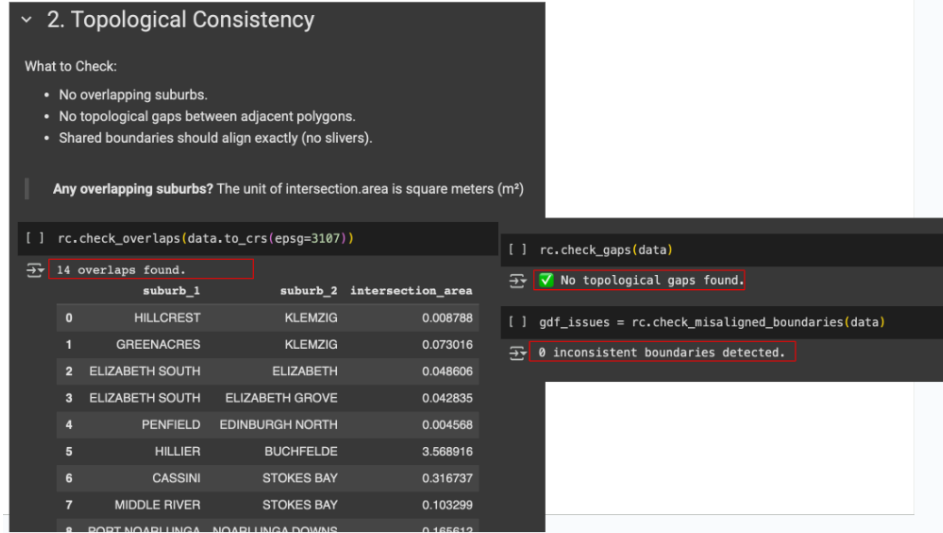


Figure 2: Topological Consistency Checks

3.3 Semantic Accuracy

Semantic validation ensures attribute data is meaningful and consistent with geometric features. The following attributes were checked:

- Suburb names: unique or consistent across MultiPolygons
- Postcode and suburb number: within valid value ranges
- Legal start date: not null and properly formatted

Results are highlighted as shown in Figure 3 that 27 suburbs with invalid or missing legal start dates (values marked as NaT, i.e., Not a Time) were detected, while neither duplicate/missing suburb names nor missing postcodes/suburb numbers were found.

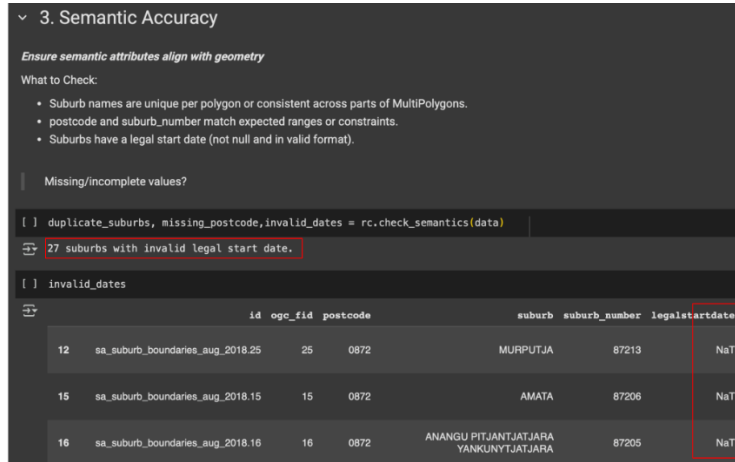


Figure 3: Semantic Accuracy Checks

3.4 Implementation and Reusability

All validation rules were implemented and encapsulated as individual Python functions as shown in Figure 4, designed for modular reuse and prepared for future client-end employment. For this suburb boundary case, these include:

- `check_geometries`
- `check_areas`
- `check_overlaps`
- `check_gaps`
- `check_misaligned_boundaries`
- `check_semantics`

To use these checks we simply follow the procedure: 1. Load and parse the GeoJSON file into a GeoDataFrame. 2. Reproject the data to a magnitude meaningful CRS (e.g., EPSG:3107 in meters) as needed. 3. Call functions for automated validation.

```

└─ Baozjinxz
> def check_overlaps(gdf: gpd.GeoDataFrame):...

└─ Baozjinxz
> def check_areas(gdf: gpd.GeoDataFrame, reproject_epsg=3187):...

└─ Baozjinxz
> def check_geometries(gdf: gpd.GeoDataFrame):...

# --- Check for topological gaps ---
└─ Baozjinxz
> def check_gaps(gdf: gpd.GeoDataFrame):...

# --- Check for semantic consistency ---
└─ Baozjinxz
> def check_semantics(gdf: gpd.GeoDataFrame):...

# --- Check for misaligned shared boundaries ---
└─ Baozjinxz
> def check_misaligned_boundaries(gdf: gpd.GeoDataFrame):...

```

Figure 4: Validation Functions Encapsulated in Python

3.5 Summary

We have presented a structured approach to validating spatial data, focusing on geometric validity, topological consistency, and semantic accuracy. Using modular Python functions, the analysis revealed:

- **Geometric integrity:** All polygons were valid and properly bounded.
- **Topological issues:** While no gaps or boundary misalignments were found, 14 overlapping suburb areas were detected and flagged for further review.
- **Semantic errors:** 27 records were found with invalid or missing legal start dates, indicating the need for attribute-level data cleaning.

The validation framework proved effective for quickly identifying issues and ensuring data readiness for downstream spatial analyses. Yet future work on integrating a complete set of validation rules pertaining to various geospatial datasets is compulsory, which would be discussed in the next section.

4 Proposal for semester 2

In the second semester, we plan to develop and deploy a fully functional geospatial data validation system that automates the processing of geospatial datasets, applies rule-based quality checks, and generates interpretable validation reports. Our plan includes six major development steps and one key extension focusing on interactive rule selection and user-defined rule integration.

Step 1: Modularizing Validation Rules into Dockerised Units

In this step, each validation rule will be implemented as an independent Python function that accepts a GeoDataFrame as input and returns a subset of invalid records along with descriptive error messages. For example, Figure 5 `check_geometry_validity` will return features where the geometry is invalid, while `check_overlap_buildings` will identify overlapping polygons using spatial indexing techniques.

```
def check_geometry_validity(gdf):  
    failed = gdf[~gdf.is_valid]  
    return failed, "Geometry is not valid"  
  
def check_overlap_buildings(gdf):  
    # Use STRtree to detect overlapping features  
    return failed_buildings, "Overlapping buildings detect"
```

Figure 5: Step 1 - Modularizing Validation Rules into Independent Units

Additionally, based on client's feedback, our system should make sure that users can check the rules they wish to apply, or use the default "Apply All" option. Thus, to ensure platform independence and runtime consistency, we plan that all internal validation rules will be encapsulated within a dedicated Docker container. This container will store all validated rule scripts and expose them via a callable interface. During execution, the main system will pass datasets to the container via standardized APIs.

Step 2: Pipeline for Rule Execution

After our all rule functions are modularized, we will move onto Step 2: implementing a rule execution controller which applies each selected rule to the input dataset. This pipeline function, such as Figure 6 `run_all_rules(gdf, rules)`, will integrate the validation results into a same output structure, describes each failed record with the rule type and its error.


```
def run_all_rules(gdf, rules):
    failed_list = []
    for rule_func in rules:
        failed, reason = rule_func(gdf)
        failed["error_type"] = reason
        failed_list.append(failed)
    return pd.concat(failed_list)
```

Figure 6: Step 2 - Pipeline for Rule Execution

Step 3: Support for Multiple Dataset Types

To make sure our system could support variety datasets—such as buildings, roads, and sub boundary, as shown in Figure 7. Our system will dynamically identify the dataset type and match the appropriate set of rules. Each rule will be tagged with one or more data types, and selection will be based on metadata inspection or filename parsing.

```
building_rules = [check_geometry_validity, check_overlap_buildings]
road_rules = [check_geometry_validity, check_dangling_lines]

def validate(dataset, dtype):
    if dtype == "building":
        return run_all_rules(dataset, building_rules)
```

Figure 7: Step 3 – Support Multiple Dataset Types

Step 4: Automate with Airflow

Then we plan to automate the validation workflow by using Apache Airflow. Once a new dataset is uploaded, a DAG (Directed Acyclic Graph) will be triggered to execute the appropriate rule pipeline, as shown in Figure 8. The Airflow DAG will run containerized validation tasks, storing the results in flat files and registration databases, and optionally generating output for downstream systems. This implementation enables continuous, unattended data monitoring and simulates a production-scale extraction pipeline.

```

from airflow import DAG
from airflow.operators.python_operator import PythonOperator

def validate_task():
    from validate_main import validate
    df = gpd.read_file("uploaded_data.geojson")
    result = validate(df, "building")
    result.to_csv("validation_result.csv")

dag = DAG(...) # scheduled trigger
task = PythonOperator(task_id='validate_building', python_callable=validate_task)

```

Figure 8: Step 4 – Automate with Airflow

Step 5: Output Summary Report and Visual Map

After those above steps, our system will output validation results in both structured (e.g., CSV, GeoJSON) and visual formats. Using Streamlit, users can explore results on an interactive map with invalid features highlighted and error details available on click. This promises transparency, supports data correction workflows, and ensures that validation feedback is accessible to both technical and non-technical users.

Step 6: Write Results into Registry

All validation outcomes will be recorded in a central SQLite-based registry, which is like a database. Each entry will include the feature ID, dataset type, rule name, error message, and timestamp.

Step 7: User-Selectable and Custom Rules

As we said in Step 1, we will support both rule selection and user-defined rule. After dataset uploading, users will be presented with a list of compatible rules and may select a subset or run all. If users want to define their own customized rules, they may upload Python scripts that define a `check_custom(gdf)` function conforming to our rule interface. Uploaded scripts will be sandboxed and executed inside a separate Docker container, ensuring that untrusted code cannot affect our main system. The container will mount the user-provided script and dataset in a read-only fashion, run the rule, and return results in standard format.

5 Timeline

Below is the proposed development schedule for Semester 2. Tasks are aligned with the seven implementation steps described above.

Table 1: Project Timeline and Task Breakdown

Timeline	Task Description	Key Actions	Assigned Step(s)
Week 1–2	Data preparation: build-ings, roads, POIs	Collect, clean, and inspect datasets	Preparation
Week 3–6	Modular rule development + containerisation	Implement core rules; Dockerise rule logic	Step 1
Week 6–8	Pipeline logic + rule-type mapping + Docker testing	Build rule controller, support dataset types	Step 2, 3
Week 8–10	Airflow DAGs + Streamlit output	Automate validation; implement visualisation	Step 4, 5
Week 10–12	Validation registry + custom rule interface	Store results; enable user-defined rules	Step 6, 7

This schedule Table 1 builds the system step by step, making sure each part is tested and ready before moving on. By adding containerization and user settings early, we plan to make the system easier to deploy and more flexible to use. Each stage connects smoothly to the next, with testing built in along the whole process.

6 Contribution Statement

Table 2: Individual contribution factors for each team member.

Group Member	ContributionFactor
Lei Cao	0.20
Xiaohan Deng	0.20
Xuanyi Zhang	0.20
Yaojin Wu	0.20
Zhizhou Zhu	0.20
Total	1.00

Each member contributed equally to the project across research, implementation, and report writing as shown in Table 2 in detail.

References

- Ariza López, F. e. a. (2020). Geospatial data quality (iso 19157-1): Evolve or perish. *Revista Cartográfica*, (100):129–154.
- Asghari, A., Kalantari, M., and Rajabifard, A. (2020). A structured framework for 3d cadastral data validation – a case study for victoria, australia. *Land Use Policy*, 98:104359.

- Bellet, M., Dufourd, J., and Fuchs, L. (2011). Rule-based transformations for geometric modelling. *Computers & Graphics*, 35(3):614–623.
- Butler, H., Daly, M., Doyle, A., Gillies, S., Hagen, S., and Schaub, T. (2016). The geojson format. RFC 7946.
- Dang, P., Zhu, J., Dang, C., and Zhang, H. (2025). Semantic-driven parametric 3d geographic scene modeling: Integrating knowledge graphs and large language models. *Environmental Modelling & Software*, 188:106399.
- Devendran, A. and Lakshmanan, G. (2014). A review on accuracy and uncertainty of spatial data and analyses with special reference to urban and hydrological modelling. *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, II-8:171–178.
- Government of South Australia, Department of Planning, T. and Infrastructure (2018). Sa suburb boundaries - august 2018 [data set]. <https://data.aurin.org.au/dataset/sa-govt-dpti-sa-suburb-boundaries-aug-2018-na>. AURIN - The Australian Urban Research Infrastructure Network. Accessed May 18, 2025.
- Migliorini, S. e. a. (2016). Towards massive spatial data validation with spatialhadoop. In *Proc. 3rd ACM SIGSPATIAL Workshop on Analytics for Big Geospatial Data*, pages 27–34.
- Nguyen, K.-S., Nguyen, T.-V., Ngo, H.-H., and Nguyen, D.-B. (2025). Application of large language models in geographic map analysis and visualization. *Unpublished or preprint*.
- Rajabifard, A., Feeney, M., and Williamson, I. (2003). Spatial data infrastructures: Concept, sdi hierarchy, and future directions. *Geomatics Research Australasia*, 80:1–14.
- Shojaei, D., Rajabifard, A., Kalantari, M., and Bishop, I. (2017). Geometrical data validation in 3d digital cadastre: A case study for victoria, australia. *ISPRS Int. J. Geo-Inf.*, 6(11):337.